

# アーティファクトを用いたシミュレーションモデルの作成

— Claude Code によるブラウザ内シミュレーションの技術チュートリアル —

河合勝彦\*

2026 年 7 月 12 日

## 概要

本稿は、Claude Code の「アーティファクト」機能を用いて、ブラウザ内で完結するインタラクティブ・シミュレーションを構築するための技術チュートリアルである。まずアーティファクトの機能・発動条件・技術的制約を整理し、設計を支援する二つのスキル（`artifact-design`・`dataviz`）の役割を説明する。次に、制作過程の実例として、(1) 連続時間の状態遷移と採点関数をもつ遊戯的シミュレータ「炭火職人」（うなぎ蒲焼きシミュレータ）、(2) 標準仕様のシェリング分居モデルのライブ実行環境、の二つを取り上げ、設計判断・実装・検証の全過程を示す。さらにシェリングモデルについては、アーティファクトと同一アルゴリズムの再現実行により得た分析結果（許容閾値  $\tau = 0.3$  における分居の創発、 $\tau$  と分居度の非線形関係、収束時間の急増）を報告する。

## 目次

|     |                      |   |
|-----|----------------------|---|
| 1   | はじめに                 | 2 |
| 2   | アーティファクトの機能と使い方      | 2 |
| 2.1 | Claude Code とは       | 2 |
| 2.2 | アーティファクトとは           | 3 |
| 2.3 | 発動条件と基本ワークフロー        | 3 |
| 2.4 | 技術的制約                | 4 |
| 2.5 | 設計を支援する二つのスキル        | 4 |
| 3   | 事例 1：蒲焼きシミュレータ「炭火職人」 | 5 |
| 3.1 | 題材とデザインプラン           | 5 |
| 3.2 | モデル：状態変数と更新則         | 5 |
| 3.3 | 実装のポイント              | 7 |
| 3.4 | 評価画面                 | 7 |
| 4   | 事例 2：シェリング分居モデル      | 9 |

---

\* 名古屋市立大学大学院経済学研究科 [kkawai@econ.nagoya-cu.ac.jp](mailto:kkawai@econ.nagoya-cu.ac.jp)

|     |              |    |
|-----|--------------|----|
| 4.1 | モデル仕様        | 9  |
| 4.2 | 実装           | 10 |
| 4.3 | 可視化設計とパレット検証 | 11 |
| 4.4 | 分析結果         | 12 |
| 4.5 | 再現性の担保       | 15 |
| 5   | 議論           | 15 |
| 5.1 | 教育・研究上の含意    | 15 |
| 5.2 | 限界と対処        | 15 |
| 6   | おわりに         | 15 |
| A   | 公開アーティファクト一覧 | 16 |

## 1 はじめに

エージェント・ベース・モデル (agent-based model; ABM) の研究・教育において、「モデルに触れられる」デモンストレーションの価値は大きい。パラメータを動かした瞬間にマクロな帰結が変わる体験は、数式やソースコードの静的な提示では代替できない。しかし従来、この種のデモを配布するには、NetLogo 環境の導入、Web サーバの運用、あるいは JavaScript フレームワークの学習といった準備コストが必要であった。

Claude Code (Anthropic 社が提供するコーディングエージェント、第 2 節で詳述) のアーティファクト (Artifact) 機能は、この準備コストを大幅に引き下げる。対話的に仕様を伝えるだけで、自己完結した HTML アプリケーションが `claude.ai` 上の固定 URL に公開され、ブラウザさえあれば誰でも (共有設定後に) 実行できる。本稿では、この機能を用いたシミュレーション構築の技術的要点を、実際の制作過程に即して解説する。

本稿の構成は次のとおりである。第 2 節でアーティファクトの機能と制約を整理する。第 3 節では第一の事例として、連続時間の状態遷移をもつ遊戯的シミュレータの設計と実装を示す。第 4 節では第二の事例として、シェリング分居モデルの標準仕様による実装と、その分析結果を報告する。第 5 節で教育・研究上の含意と限界を論じる。

## 2 アーティファクトの機能と使い方

### 2.1 Claude Code とは

アーティファクトの説明に先立ち、その土台である Claude Code 自体について述べる。Claude Code は Anthropic 社が提供する **コーディングエージェント** (coding agent) である。ここでの「エージェント」は本稿の主題である ABM のエージェントとは別義で、大規模言語モデル (LLM) を中核に、ファイルの読み書き・シェルコマンドの実行・Web 検索といった **ツール** を自律的に呼び出しながら、与えられた目標を達成するまで作業を続けるソフトウェアを指す。

従来の対話型 AI（チャットボット）との本質的な違いは、応答が「テキストの生成」で完結しない点にある。チャットボットはコードを提示できても、それを実行するのは利用者の仕事である。これに対し Claude Code は、利用者が自然言語で目標を伝え、作業計画を立て、コードを書き、それを実際に実行して結果を観察し、エラーや仕様とのずれがあれば自ら修正する——という「編集・実行・検証」のサイクルを、逐次の指示を待たずに繰り返す。本稿の制作過程でも、HTML/JavaScript の実装だけでなく、配色の検証スクリプトの実行（第 4.3 節）、Node.js によるパラメータスイープ（第 4.4 節）、そしてアーティファクトとしての公開までが、同一エージェントの一連のツール呼び出しとして行われた。

Claude Code は端末のコマンドラインツールのほか、デスクトップアプリ、Web ブラウザ（`claude.ai/code`）、IDE 拡張として利用できる。本稿で扱うアーティファクトは、このエージェントが備えるツールの一つ（生成した HTML を `claude.ai` 上に公開する機能）であり、後述するスキルは、エージェントが特定の種類の作業に着手する前に読み込む指針書である。すなわち本稿のワークフロー——対話による仕様確定、設計、実装、検証、公開、分析——は、すべて単一のコーディングエージェントとの対話の中で完結している。

## 2.2 アーティファクトとは

アーティファクトは、Claude が生成した HTML または Markdown ファイルを `claude.ai` 上の Web ページとしてレンダリングし、固定 URL で配信する仕組みである。重要な性質は以下のとおり。

- **既定で非公開** 公開直後は作成者本人のみが閲覧できる。他者との共有はページの共有メニューから作成者が明示的に行う。
- **固定 URL と更新** 同一ファイルパスから再公開すると同じ URL の内容が更新される。バージョンにはラベルを付与でき、履歴をたどれる。
- **静的配信** サーバサイドの計算資源は付かない。動的な挙動はすべてクライアント（閲覧者のブラウザ）の JavaScript が担う。

## 2.3 発動条件と基本ワークフロー

アーティファクトは自動的に作られるものではなく、(a) 利用者が明示的に依頼した場合（「デモを作って」「Web ページにして」）、(b) 端末のテキスト出力よりページとして見せる方が明らかに適切だと Claude が判断した場合に、ツール呼び出しとして発動する。シミュレーション構築の典型的なワークフローは次のとおりである。

1. **仕様の対話的確定** モデルの状態変数・更新規則・操作したいパラメータ・観察したい指標を自然言語で伝える。「標準的な仕様で」という指示も有効に機能する（第 4 節の事例）。
2. **デザインプランの提示** Claude が配色・タイポグラフィ・レイアウトの計画を明示する（`artifact-design` スキル、後述）。

3. **実装** 単一の HTML ファイルとして CSS・JavaScript を含めて書き下ろす。
4. **公開** Artifact ツールで URL が発行される。修正依頼は同じ URL への再公開として反映される。

なお、本稿に掲載する実行画面（図 1～図 6）はすべて、公開済みアーティファクトと同一の HTML をブラウザで実行して撮影したものである。

## 2.4 技術的制約

シミュレーション設計上、特に重要な制約は三つある。

**完全な自己完結（CSP）** 厳格な Content Security Policy により、外部ホストへの一切の通信（CDN のライブラリ、Web フォント、外部画像、`fetch/XHR/WebSocket`）が遮断される。したがって D3.js 等を使う場合はインライン化が必要であり、実務上は素の JavaScript と Canvas API で書くのが素直である。データも HTML 内に埋め込む。

**サーバサイド計算の不在** Python カーネルや NetLogo エンジンへの接続はできない。モデルは JavaScript へ移植してブラウザ内で実行するか、重い計算は事前に別環境で実行し、結果（軌道データ）を埋め込んだ「再生ビューア」として設計する。規模の目安として、Canvas 2D 描画で数千～1 万エージェント、WebGL の点描画に切り替えれば数十万エージェントまで実用的である。

**状態の非永続性** ページを再読み込みすると初期状態に戻る。再現性はシード付き乱数で担保する（第 4 節）。

このほか、閲覧者の OS 設定（ライト／ダークテーマ）に応じてページが表示されるため、配色は CSS カスタムプロパティ（トークン）として定義し、両テーマ分を用意するのが原則である（リスト 1）。

リスト 1 テーマ・トークンの定義パターン（抜粋）

```
:root{ --surface:#fcfcfb; --ink:#0b0b0b; --sA:#2a78d6; }
@media (prefers-color-scheme: dark){
  :root{ --surface:#1a1a19; --ink:#ffffff; --sA:#3987e5; }
}
:root[data-theme="light"]{ /* viewer toggle overrides */ }
:root[data-theme="dark"]{ /* both directions */ }
```

Canvas 描画はこのトークンを `getComputedStyle` で読み取り、テーマ切替イベント（`matchMedia` の変化と `data-theme` 属性の `MutationObserver`）で再描画する。

## 2.5 設計を支援する二つのスキル

制作品質は二つのスキル（Claude Code に読み込ませる指針書）に支えられている。

**artifact-design** ページ制作前に必ず読み込む設計指針である。依頼を「実用的な扱い」（文書・計画書）と「エディトリアルな扱い」（ランディングページ・ゲーム・共有されるツール）に読み分け、後者では定型的な出力（いわゆる AI 的デザイン）を避けて題材固有の視覚言語を構築することを要求する。コードを書く前に、配色（4～6 色の名前付きパレット）・書体（役割別に 2 種以上）・レイアウト構想からなる「デザインプラン」を明示する手続きが定められている。

**dataviz** チャートの一つでも含む場合に読み込む作図規範である。特徴は「色は計算で検証する」という原則にあり、付属の検証スクリプトが (1) 明度帯域, (2) 彩度下限, (3) 色覚多様性 (CVD) 下での分離度 (Machado 2009 シミュレーションによる  $\Delta E$ ), (4) 背景とのコントラスト比を機械的に判定する。本稿の事例でも全系列色をこの検証に通した (第 4.3 節)。このほか、二重軸の禁止, 2 系列以上での凡例必置, データ色を文字に使わない, 等の規則が定められている。

### 3 事例 1：蒲焼きシミュレータ「炭火職人」

#### 3.1 題材とデザインプラン

第一の事例は、浜名湖うなぎの蒲焼きを炭火で焼き上げる遊戯的シミュレータである<sup>1)</sup>。「インタラクティブなデモ」という短い依頼から出発し、リポジトリの既存コンテンツ（浜名湖うなぎ紹介サイト）に題材を接地させた。遊戯的な題材だが、内部は連続時間の状態遷移モデルであり、ABM 的なシミュレータ構築の基本要素（状態変数・更新則・操作・評価関数・可視化）をすべて含む。

デザインプランでは「夜の焼き場」という単一の視覚世界に振り切ることを選択し、例外規定（意図的な単一テーマ）を適用して墨色の地に熾火色をアクセントとするダークテーマのみで構成した。タイポグラフィは見出し・引用に明朝，UI にゴシック，格言「串打ち三年，裂き八年，焼き一生」を縦書き（`writing-mode: vertical-rl`）で配置している。

図 1 に開始直後の画面を示す。中央の焼き台は Canvas 2D による動的描画で、熾火の明滅・煙・炎がリアルタイムに変化する。その下の HUD（Head-Up Display：ゲーム画面に常時表示される状態情報表示。航空機の同名装置に由来）には炭の火力、両面（身側・皮側）の焼き加減、タレ回数のメーターが並び、いま炭側（下向き）を向いている面をバッジで示す。操作は「団扇であおぐ」「裏返す」「タレにくぐらせる」「仕上げる」の 4 ボタンに集約されている。この時点で「タレにくぐらせる」が無効化されているのは、次項で述べるフェーズゲートによる。メーター上部のヒント行は、内部状態に応じた助言（優先度付きの規則群として実装）を表示し、チュートリアルなしでも遊べる誘導の役割を担う。

---

1) 公開 URL: <https://claude.ai/code/artifact/100b5bc4-c60e-42cd-9e19-f9277dbb6685>（既定で非公開）。



図1 「炭火職人」開始直後の画面。Canvas 描画の焼き台（熾火・網・串打ちした鰻），HUD（炭の火力・両面の焼き加減・タレ回数），4 つの操作ボタン。白焼きが完了するまで「タレにくぐらせる」は無効化されている。

### 3.2 モデル：状態変数と更新則

プレイヤーが操作するのは「団扇であおぐ」「裏返す」「タレにくぐらせる」「仕上げる」の 4 ボタンのみである．内部状態は火力  $h(t) \in [30, 100]$ ，両面の焼き加減  $D_i(t)$  ( $i \in \{\text{身側}, \text{皮側}\}$ )，焦げ量  $C_i(t)$ ，タレ回数  $n$ ，タレ湿潤の残り時間  $w(t)$  である．更新則は毎フレーム（時間刻み  $\Delta t$ ）：

$$\frac{dh}{dt} = \kappa (h^* - h), \quad \kappa = 0.055, h^* = 46 \quad (1)$$

$$\frac{dD_d}{dt} = \rho(h) \beta(w), \quad \rho(h) = 13 \cdot \frac{h - 30}{70}, \quad \beta(w) = \begin{cases} 1.25 & (w > 0) \\ 1 & (w \leq 0) \end{cases} \quad (2)$$

$$\frac{dC_d}{dt} = 0.85 \rho(h) \mathbf{1}[D_d > 100] \quad (3)$$

ここで  $d$  は炭側（下）を向いている面を表し，「裏返す」操作が  $d$  を切り替える．「団扇」は  $h \leftarrow \min(100, h + 16)$  の離散ジャンプである．式 (1) の時定数は約 18 秒であり，放置すれば火力は熾火の平衡値 46 に緩和する．プレイヤーは団扇による離散介入で  $h$  を管理し，式 (2) の焼成速度を制御する—これが本作の中核的なトレードオフ（高火力は速いが式 (3) の焦げリスクを増す）を生む．

タレは両面が白焼き基準 ( $D_i \geq 50$ ) に達するまで解禁されないフェーズゲートをもち，「仕上げる」で評価関数

$$S = 100 - \sum_i b(D_i) - 1.2 \sum_i C_i - P(n), \quad b(D) = 0.9(70 - D)_+ + 0.8(D - 95)_+ \quad (4)$$

が計算される ( $(x)_+ = \max(x, 0)$ )． $P(n)$  はタレ回数のペナルティで  $n = 2, 3$  を最適とする．焼き加減の理想帯  $[70, 95]$ ・焦げ係数・タレ最適回数という 3 つの評価軸が互いに緊張関係にあるため，単調な攻略法が存在しない．

### 3.3 実装のポイント

描画は Canvas 2D で行い，`requestAnimationFrame` ループ内で状態更新と描画を分離した．熾火の明滅は決定的擬似乱数と正弦波の合成，煙・炎は簡易パーティクル系（生成率を  $h$  と  $w$  に比例させる）で表現している．タレが炭に落ちて炎が上がる演出は，実際の蒲焼きの物理に対応した「モデルに正直な」エフェクトである．アクセシビリティとして `prefers-reduced-motion` 設定時はパーティクルを停止し，効果音（WebAudio でバンドパスした白色雑音，音量は  $h$  に追従）は既定オフのトグルとした．

図 2 は，両面の白焼きを終えてタレに 1 度くぐらせた直後の局面である．鰻は鮎色の照りをまとい（タレ回数と湿潤状態に応じてグレーズと光沢の描画を重ねている），ヒント行は「タレが炭に落ちて炎が上がる」ことを告げる．HUD からは両面の焼き加減がともに 70 に達し，皮側が炭側を向いていることが読み取れる．

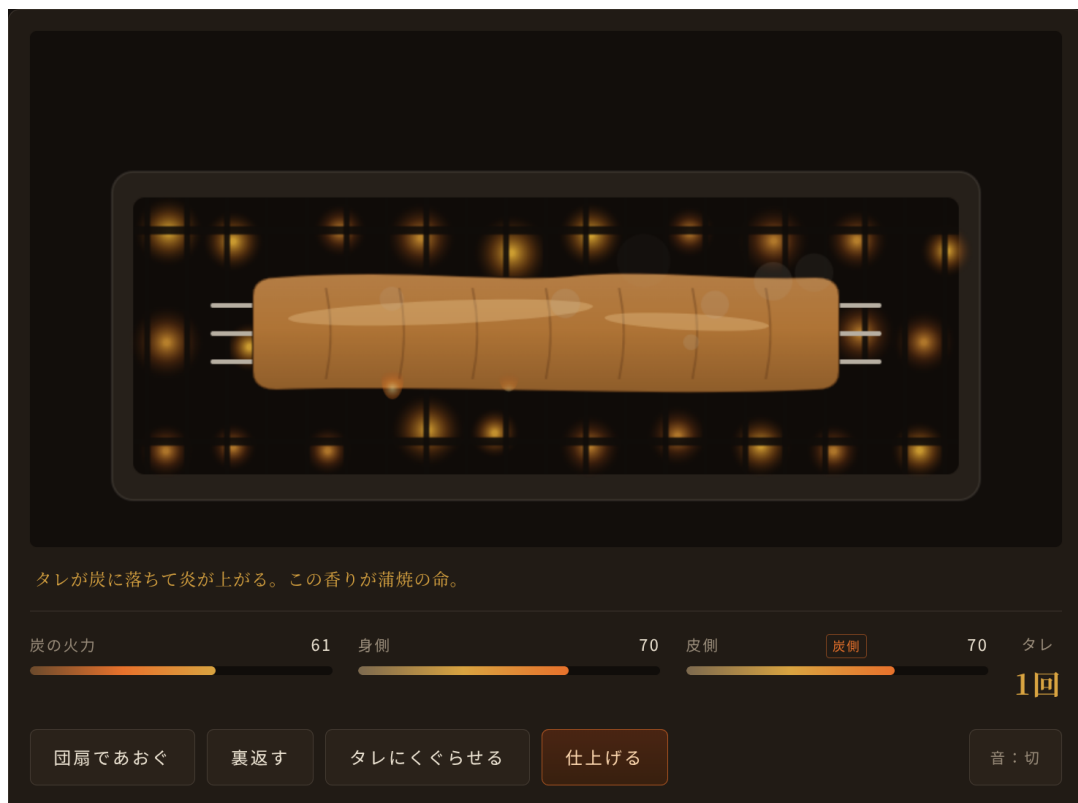


図2 タレ焼きの局面。両面の白焼き（各70）を終えてタレを1度まとった状態。タレの湿潤中（ $w > 0$ ）は焼成が1.25倍に加速し、炭に落ちたタレが炎を上げる。HUDの「炭側」バッジは皮側が下を向いていることを示す。





図3 評価画面。身側 82・皮側 82・タレ 2 回・焦げ僅少で評点 100・番付「匠」。重箱と鰻は CSS のみで描画され、焼き上がり（生焼け・良好・焦げ）に応じて色が変わる。

### 3.4 評価画面

「仕上げる」を押すと式 (1)–(3) の履歴に基づいて評価関数が計算され、朱漆の重箱に盛られた結果画面が表示される（図 3）。図の例は、身側 82・皮側 82・タレ 2 回・焦げ僅少で評点 100（番付「匠」）を得た一枚である。結果画面には評価の内訳（両面の焼き加減・タレ回数・焦げ量）が明示されるため、プレイヤーは何が加点・減点の要因だったかを事後的に確認し、次の一枚に反映できる。焼き加減の理想帯  $[70, 95]$  はタレの湿潤ブースト（1.25 倍）を考慮すると意外に狭く、仕上げの数秒を誤るだけで焦げが急速に蓄積する—この時間的な緊張感が本作の遊戯性の中心である。

## 4 事例 2：シェリング分居モデル

### 4.1 モデル仕様

第二の事例は Schelling (1971) の分居モデルの標準仕様によるライブ実行環境である<sup>2)</sup>。依頼は「モデル仕様は標準的なもので」であり、実装した仕様は次のとおり。

- $L \times L$  正方格子，周期境界（トーラス），ムーア近傍（8 近傍）。
- 2 グループのエージェント。全セルから空き地率  $v$  分を除き，指定比率で配分してランダム配置。
- 満足条件：占有近傍セルに占める同類の割合が許容閾値  $\tau$  以上（占有近傍が 0 の場合は満足）。

2) 公開 URL: <https://claude.ai/code/artifact/9ac47541-7715-48f8-9513-39ef5e4c9772>（既定で非公開）。

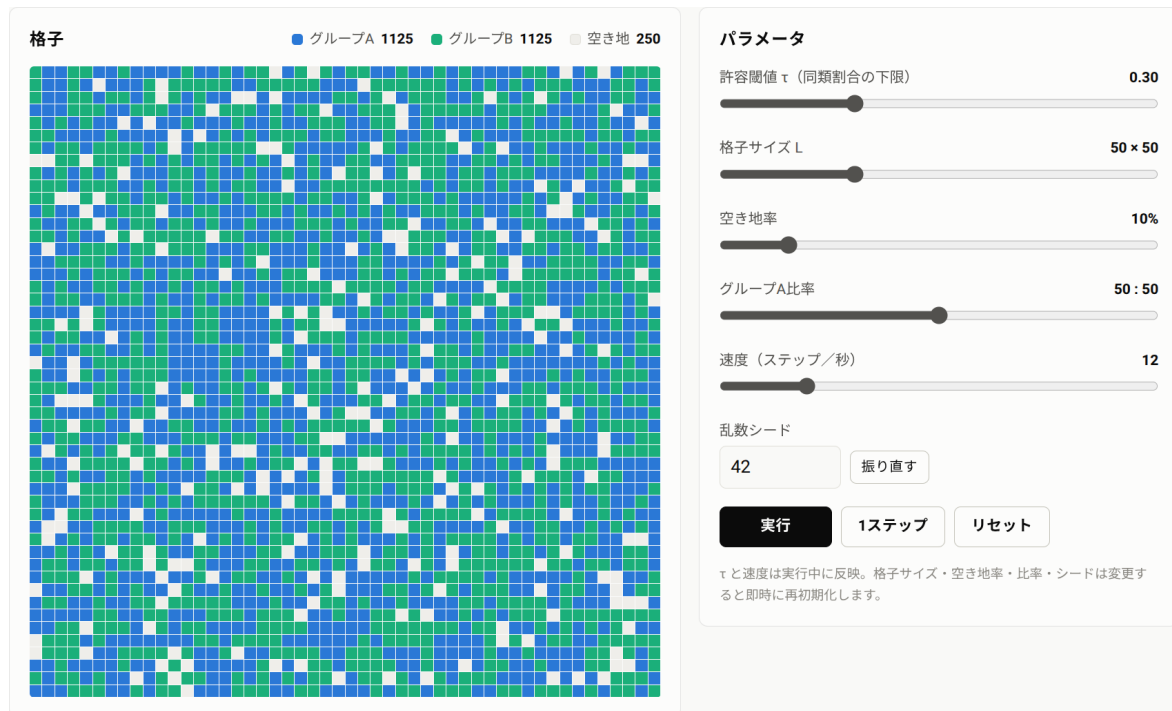


図4 シェリング分居モデルの初期画面 ( $L = 50$ ,  $v = 0.10$ , 50:50, シード 42, ライトテーマ表示). 左: ランダム混住の格子 (平均同類近傍率 49.4%) と凡例 (グループ別個体数). 右: パラメータパネルと実行制御.

- 更新規則: 各ステップ開始時に不満エージェントを全列挙し, ランダム順に, 一様ランダムに選んだ空き地へ逐次移動 (非同期更新).
- 停止条件: 不満エージェントが 0 になった時点で収束.

マクロ指標は, 平均同類近傍率  $\bar{s}$  (占有近傍を 1 以上もつエージェントの同類割合の平均; 分居の程度) と満足率である. UI では  $\tau$  と実行速度を実行中に変更でき, 格子サイズ ( $20 \leq L \leq 120$ )・空き地率・グループ比・乱数シードは変更時に即時再初期化される.

図 4 に初期状態の画面を示す (シード 42). 左の格子パネルには 2 グループ (青・アクア) のエージェントと空き地が描画され, 凡例に個体数 (1,125・1,125・250) を常時表示する. セル間には 1px の地色の隙間 (surface gap) を設け, 色の塗り分けだけに頼らない判別を助けている. 右のパラメータパネルには 6 つの操作 ( $\tau$ ・格子サイズ・空き地率・グループ比・速度・乱数シード) と実行制御 (実行・1 ステップ・リセット) が並ぶ.  $\tau$  のような感度分析の主役はスライダーを動かした瞬間に反映され, 格子の作り直しを要する構造パラメータは変更と同時に再初期化される—この区別を UI 上の注記で明示している. 初期表示の平均同類近傍率 49.4% は, ランダム混住の理論値 0.5 とよく一致する.

## 4.2 実装

再現性の要は、シード付き乱数生成器 `mulberry32` である (リスト 2)<sup>3)</sup>。初期配置のシャッフルと移動先の選択がすべてこの生成器を通るため、同じシードと設定なら同じ軌道が完全に再現される。

リスト 2 シード付き乱数生成器 `mulberry32`

```
function mulberry32(seed){
  var a = seed >>> 0;
  return function(){
    a |= 0; a = (a + 0x6D2B79F5) | 0;
    var t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}
```

1 ステップ (スイープ) の中核はリスト 3 のとおり。空き地リストに対する「選んだ空き地を移動元セルで置き換える」スワップ操作により、空き地の一様抽選を  $O(1)$  で維持している。格子は `Int8Array`、近傍はトラス上のオフセットを事前計算した `Int32Array` で保持し、 $L = 120$  (14,400 セル) でも毎秒 60 ステップの実行が可能である。

リスト 3 スイープ (1 ステップ) の中核部

```
// enumerate unsatisfied agents at sweep start, shuffle,
// then move each to a uniformly random vacant cell
var moves = 0;
for(var m = 0; m < unsatList.length; m++){
  if(vacant.length === 0) break;
  var from = unsatList[m];
  var vi = (rng() * vacant.length) | 0;
  var to = vacant[vi];
  vacant[vi] = from; // vacated cell replaces the chosen vacancy
  grid[to] = grid[from]; grid[from] = 0;
  moves++;
}
```

## 4.3 可視化設計とパレット検証

可視化は `dataviz` スキルの手順に従った。エンティティごとに固定順で色を割り当て (グループ A = 青, グループ B = アクア, 平均同類近傍率 = 黄, 満足率 = 緑; 空き地はデータでない)

---

3) Tommy Ettinger 作の 32 ビット状態の生成器 (<https://gist.github.com/tommyettinger/46a874533244883189143505d203312c>). JavaScript 標準の `Math.random()` にはシード指定機能がないため、シード付き乱数を 1 行数十文字で書ける手軽さから事実上の定番となっている。

め無彩色), 全系列を検証スクリプトにかけた。結果の要点:

- 格子の 2 色 (青・アクア): CVD 分離  $\Delta E = 73.6$  (ライト面) /  $69.8$  (ダーク面) で, 目標値 12 を大きく上回り合格.
- チャートの 2 色 (黄・緑): ライト面で黄のコントラスト比 2.11 (3:1 未満の警告) → 救済条件として凡例への現在値常時表示・ツールチップ・ステータスタイルを設置. ダーク面で  $\Delta E = 10.3$  (下限帯 8-12) → 直接ラベルの併設により許容.

「警告は救済条件 (可視ラベル等の第二の情報チャンネル) とセットでのみ許容される」という規範が, 機械判定によって強制される点が本手法の特長である. このほか, 2 指標はいずれも割合 (0-1) であるため単一の縦軸 (0-100%) を共有させ, 二重軸を回避した. 時系列チャートにはクロスヘア付きツールチップを実装し, 全履歴を CSV としてクリップボードに出力するボタンを設けて外部解析への接続点とした.

#### 4.4 分析結果

以下の数値は, アーティファクトと同一のアルゴリズム・同一の乱数生成器を Node.js で再現実行して得たものであり, アーティファクト上でも同じシードを入力すれば同じ軌道が観察できる<sup>4)</sup>.

**基準実行**  $L = 50$ ,  $v = 0.10$ , グループ比 50:50,  $\tau = 0.30$ , シード 42 の基準実行では, 初期配置の平均同類近傍率は  $\bar{s}_0 = 0.494$  (ランダム混住の理論値 0.5 と整合) で, 初期満足率はすでに 82.7% である. すなわち大多数のエージェントは初期状態に満足している. それにもかかわらず, わずか 14 ステップ (延べ 1,003 回の移動) で全員満足に収束し, 最終的な平均同類近傍率は  $\bar{s} = 0.741$  に達した (図 7). 「同類が 3 割いれば満足」という穏やかな個人選好が, 74% という強い集団的分居を生む—シェリングが示した創発の核心が, そのまま再現されている.

この基準実行はアーティファクト上でもそのまま観察できる. 図 5 は収束時点の格子で, 初期のランダム混住 (図 4) から同色のクラスタが明瞭に形成され, パネル上部に収束バッジ (ステップ 14) が点灯している. 収束後は「実行」「1 ステップ」が無効化され, パラメータを変えるまで系が静止したことが操作面からも分かる. 図 6 は同時点の統計タイトルと時系列チャートである. 表示値 (平均同類近傍率 74.1%・満足率 100.0%) は, Node.js による再現実行 (図 7) と桁まで一致しており, ブラウザ上のライブ実行とオフライン再現が同一の軌道をたどることが画面上でも確認できる.

**許容閾値のスweep**  $\tau$  を変化させ, 各 10 シード (1-10)・上限 3,000 ステップで実行した結果を表 1 と図 8 に示す.

---

4) Node.js の併用は分析上の必然ではなく, バッチ実行の足場としての選択である. 基準実行のような単一の実行は, アーティファクト単体で観察から CSV 出力まで完結する. 一方, 後述の許容閾値スweep は 8 水準  $\times$  10 シードの計 80 回の実行と集計を要し, 描画と同期した対話的実行を UI 操作で繰り返して手作業で集計するのは現実的でない. 描画を切り離れた Node.js では全 80 回の実行が数秒で完了し, 結果をそのまま表 1 に整形できる. なお, 両環境の結果が桁まで一致すること (図 6) は, この再現実装の正しさの検証を兼ねている.

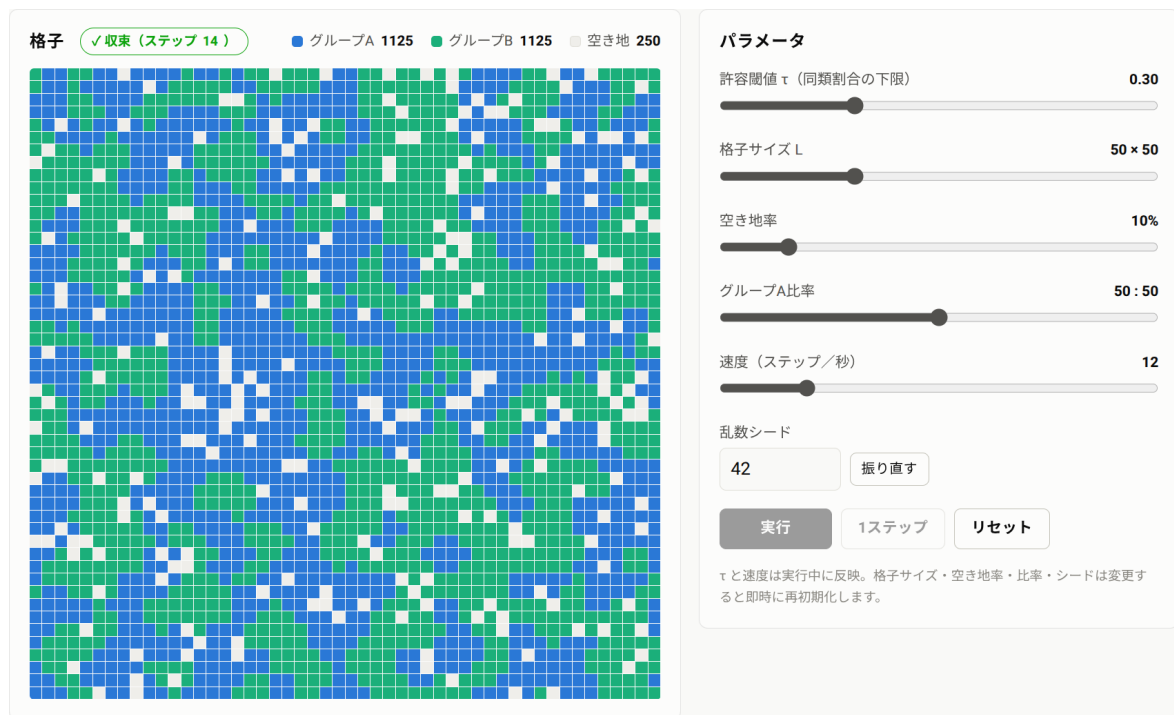


図5 収束後の格子（ステップ 14， $\tau = 0.30$ ，シード 42）．同色クラスタが形成され，収束バッジが点灯する．収束により「実行」「1 ステップ」ボタンは無効化されている．

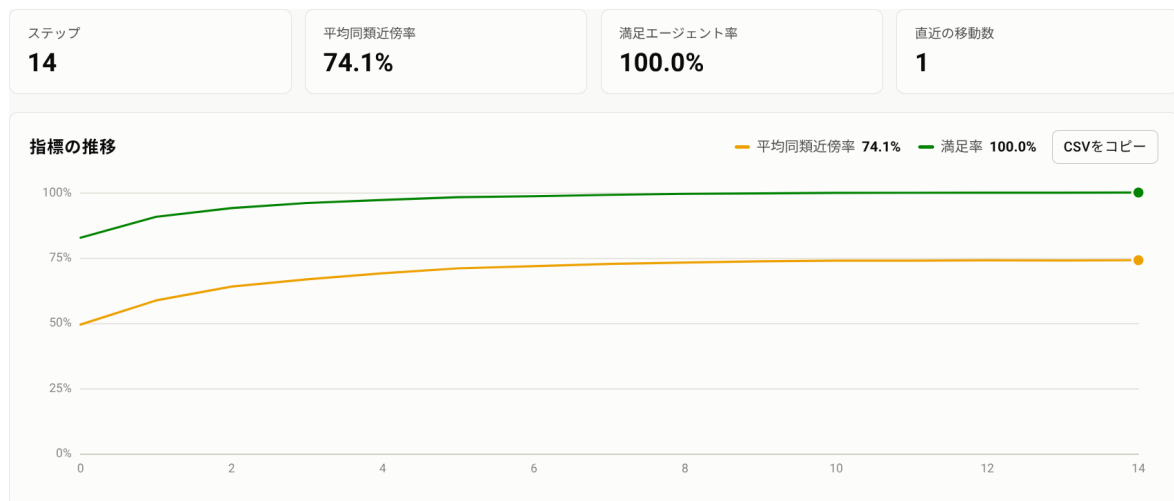


図6 収束時点の統計タイルと指標チャート．平均同類近傍率 74.1%・満足率 100.0%は再現実行（図7）と一致する．凡例には現在値が常時表示され，ライトモードで 3:1 未満となる黄系列のコントラスト警告に対する救済チャンネルを構成する（第 4.3 節）．

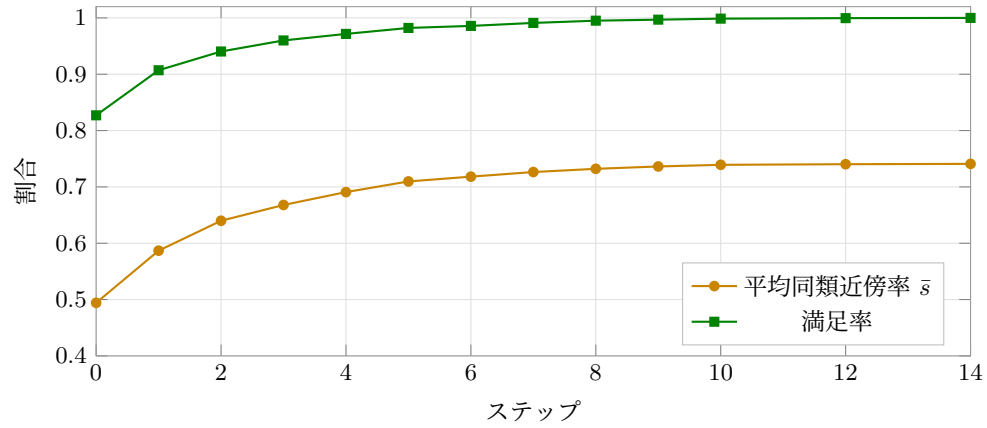


図7 基準実行の軌道 ( $L = 50$ ,  $v = 0.10$ , 50:50,  $\tau = 0.30$ , シード 42). 14 ステップで収束し, 平均同類近傍率は 0.494 から 0.741 へ上昇する.

表1 許容閾値  $\tau$  のスイープ ( $L = 50$ ,  $v = 0.10$ , 50:50, シード 1-10, 上限 3,000 ステップ)

| $\tau$ | 最終 $\bar{s}$ (平均) | 標準偏差  | 平均収束ステップ | 収束数   |
|--------|-------------------|-------|----------|-------|
| 0.10   | 0.511             | 0.006 | 1.4      | 10/10 |
| 0.20   | 0.576             | 0.014 | 6.4      | 10/10 |
| 0.25   | 0.581             | 0.015 | 6.2      | 10/10 |
| 0.30   | 0.750             | 0.011 | 12.6     | 10/10 |
| 0.40   | 0.826             | 0.010 | 16.1     | 10/10 |
| 0.50   | 0.867             | 0.006 | 17.4     | 10/10 |
| 0.60   | 0.974             | 0.001 | 131.0    | 10/10 |
| 0.70   | 0.995             | 0.001 | 207.6    | 10/10 |

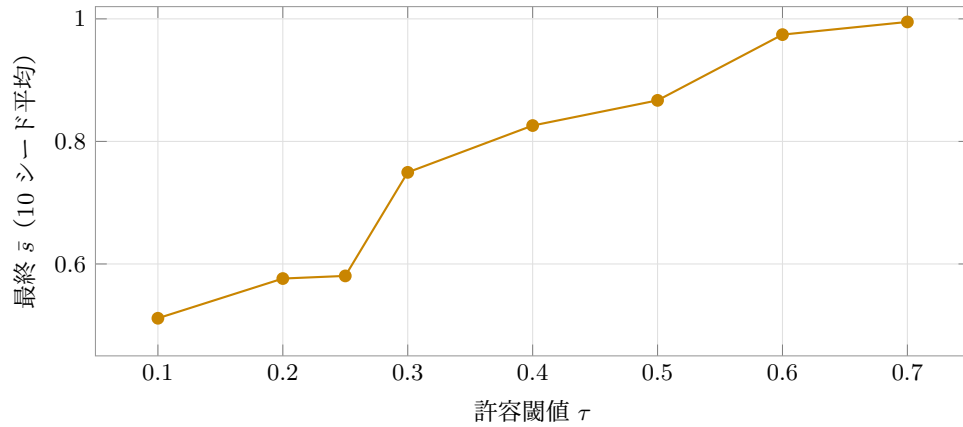


図8 許容閾値  $\tau$  と最終的な平均同類近傍率の関係.  $\tau = 0.25 \rightarrow 0.30$  で不連続に跳躍する.

三つの特徴が読み取れる。第一に、 $\tau$  と分居度の関係は単調だが強く非線形であり、特に  $\tau = 0.25 \rightarrow 0.30$  で  $\bar{s}$  が 0.58 から 0.75 へ不連続に跳躍する。これはムーア近傍の離散性による。占有近傍が 8 のとき、満足に必要な同類数は  $\lceil 8\tau \rceil$  であり、 $\tau = 0.25$  では 2 人、 $\tau = 0.30$  では 3 人と、実効閾値が整数単位で切り替わるためである。連続パラメータの背後にある離散構造が集計量に現れる好例であり、講義での議論素材に適する。

第二に、 $\tau = 0.10$  では  $\bar{s} = 0.511$  と初期値からほとんど動かない。不満者が僅少のため 1-2 ステップで収束し、分居はほぼ生じない。創発的分居には「穏やかだがゼロではない」選好水準が必要であることがわかる。

第三に、 $\tau \geq 0.60$  の高選好域では、最終分居度が 0.97 を超えて飽和する一方、収束までの平均ステップ数が 131 ( $\tau = 0.6$ )、208 ( $\tau = 0.7$ ) と一桁以上増大する。要求水準が高いほど「引っ越しの連鎖」が長く続き、ほぼ完全な棲み分け（少数の巨大クラスター）に達してようやく停止する。なお空き地率 10%の本設定では上限 3,000 ステップ内に全 run が収束したが、 $\tau$  をさらに上げれば非収束（永続的な churning）に転じることが知られている。

## 4.5 再現性の担保

以上の分析はすべて、(1) シード付き乱数による軌道の完全な決定性、(2) アーティファクトの CSV 出力 (step, mean\_like\_neighbor\_frac, satisfied\_frac, moves)、の二点によって、アーティファクト上でもオフライン解析でも同一に再現できる。この「ブラウザ内のデモが同時に再現可能な計算実験装置でもある」という性質は、教材と研究ツールの距離を縮めるものである。

# 5 議論

## 5.1 教育・研究上の含意

本稿の二事例が示すように、アーティファクトによるシミュレーション構築は (1) 配布コストが URL 一つ、(2) 実行環境がブラウザのみ、(3) シードによる再現性、という三点で教育利用に適する。講義では受講者が各自のデバイスでパラメータを操作でき、演習では CSV 出力を起点に統計解析へ接続できる。研究面では、モデルの挙動を共同研究者に伝える「動く補足資料」としての利用が現実的である。

## 5.2 限界と対処

第 2.4 節の制約から、大規模計算（数十万エージェント超、長期 run、広いパラメータスイープ）には向かない。この場合は、計算を Python (Mesa 等) や Julia で別途実行し、軌道データを埋め込んだ再生ビューアとしてアーティファクトを設計する二段構成が有効である。スイープ結果のヒートマップから特定 run の再生に遷移する構成も同じ枠組みで実現できる。また、アーティファクトは既定で非公開であり、共有には作成者の明示的な操作が必要である点、および外部データの動的取得ができない点には留意されたい。

## 6 おわりに

本稿では、Claude Code のアーティファクト機能によるシミュレーション構築を、機能解説と二つの実例（連続時間の遊戯的シミュレータ、標準仕様のシェリング分居モデル）を通じて示した。とりわけ後者では、対話による仕様確定から、検証スクリプトに裏付けられた可視化設計、シード付き乱数による再現可能な分析までの全過程が、単一の HTML ファイルと URL に収まることを確認した。ABM の教育・研究における「触れるモデル」の敷居は、確実に下がっている。

### A 公開アーティファクト一覧

- 炭火職人（うなぎ蒲焼きシミュレータ）

<https://claude.ai/code/artifact/100b5bc4-c60e-42cd-9e19-f9277dbb6685>

- シェリング分居モデル（ライブ ABM）

<https://claude.ai/code/artifact/9ac47541-7715-48f8-9513-39ef5e4c9772>

いずれも既定で非公開であり、閲覧には作成者による共有操作が必要である。

## 参考文献

- [1] Schelling, T. C. (1971) “Dynamic Models of Segregation,” *Journal of Mathematical Sociology*, 1(2), 143–186.